

晶片 Floorplanning 比賽 新手完全入門

從零開始，看懂我們的專案在做什麼、
現在卡在哪、接下來要怎麼贏

ICCAD 2026 CAD Contest — Problem C

FloorSet-Lite 賽道 | 2026-06-30

完整技術版見專案內 WINNING_STRATEGY.md

這份文件會帶你到哪

看完這份文件，就算你完全沒碰過晶片設計，也能完整回答這四個問題：

1. 這個比賽到底在做什麼？（第 1–2 章）
2. 分數怎麼算、贏的關鍵在哪？（第 3 章）
3. 我們現在有哪些做法、各自的強弱？（第 4–5 章）
4. 為什麼現在的 AI 做法是壞的、要怎麼修才能拿第一？（第 6–7 章）

最後第 8 章說明「接下來你做什麼、Claude (AI 助手) 做什麼」，附錄有名詞小辭典。

怎麼讀這份文件

每一個技術名詞，我都會先給生活比喻再講真正的意思。看到 **藍框** 是比喻、**綠框** 是重點、**紅框** 是警告、**紫框** 是實際數字例子。

第 1 章 Floorplanning 是什麼？

晶片（你手機、電腦裡的那塊小東西）裡面有很多「功能區塊」——有的負責計算、有的負責記憶、有的負責通訊。設計晶片時，其中一步就是要決定：這些區塊各自要放在晶片的哪個位置、長成什麼形狀。這一步就叫 **Floorplanning**（平面規劃）。

生活比喻：擺積木

想像你有一個固定大小的盒子，要把一堆長方形積木放進去。你要做到三件事：

- ① **塞得緊**——不要浪費空間（省晶片面積 = 省錢）
- ② **相關的積木靠近**——有些積木之間要用「電線」連起來，線越短越好
- ③ **守規矩**——有些積木有特殊規定（等一下第 2 章講）

為什麼這件事很難？

因為「擺法」的數量多到無法想像。如果有 120 個積木，可能的擺放組合大約是 10^{250} 種。

這個數字有多大？

整個宇宙的原子數量大約是 10^{80} 個。 10^{250} 是這個數字的 再乘上 10^{170} 倍。你不可能把每種擺法都試一遍，所以這是一個「在天文數字的選項裡找好答案」的難題。

這類問題在數學上叫「組合最佳化 (combinatorial optimization)」，是出了名的難。這也是為什麼這個比賽有意義——它在考驗「誰能在有限時間內，從天文數字的選項裡找到最好的擺法」。

第 2 章 比賽到底要我們做什麼？

給我們什麼（輸入）

- 一張積木清單：每個積木要多大面積。
- 連線表（netlist）：哪些積木之間要連電線、連得多重要。
- 一堆規定（約束）：見下表。

要我們交什麼（輸出）

每個積木最終的位置 (x, y) 和形狀 (寬, 高)。

五種規定（約束）— 用比喻記

規定名稱	白話比喻	違反的後果
Fixed-shape 固定形狀	這個家具尺寸不能改（像一台尺寸固定的冰箱）	硬規定：違反直接判定失敗
Preplaced 預先放置	這個家具位置被釘死（像牆上的固定插座）	硬規定：違反直接判定失敗
Soft area 軟性面積	可以變胖變瘦，但面積要保持（±1%）（像一塊可揉的黏土）	硬規定：面積差超過 1% 就失敗
Grouping 分組	這幾個要放在一起（一家人要坐同桌）	軟規定：分散會被扣分
MIB 同形	這幾個必須長得一模一樣（雙胞胎要穿同款）	軟規定：不一樣會被扣分
Boundary 靠邊	這個必須靠某一面牆（書架要靠牆放）	軟規定：沒靠到會被扣分

★ 重點：硬規定 vs 軟規定

硬規定（前三項）= 紅線，碰到就「直接失敗」（這題拿最差的分數）。

軟規定（後三項）= 扣分項，違反會讓分數變差但不會直接失敗。

這個區別很重要，因為它決定了我們的程式「哪些事絕對不能做、哪些事可以權衡」。

📦 額外的好東西：一百萬個範例答案

主辦單位除了出題，還附送了 **100 萬筆「已經解好的範例」**（每筆都是一道題 + 一個接近最佳的擺法）。這 **100 萬筆範例**，是我們能不能拿第一的關鍵武器——第 7 章會解釋為什麼。

第 3 章 分數怎麼算？（白話版）

每一道題的分數叫 **Cost (成本)**，**越低越好**。它由三個部分相乘：

$$\text{Cost} = \underbrace{(\text{接線長度} + \text{面積 表現})}_{\text{可正可負}} \times \underbrace{(\text{軟規定違規懲罰})}_{\text{1 到 7.4 倍}} \times \underbrace{(\text{速度})}_{\text{最多省 30\%}}$$

三個你必須記住的關鍵

★ 關鍵 1：分數可以低於 1（比範例答案更好）

分數是拿我們的答案跟「參考答案」比。參考答案只是「接近最佳」，**不是完美**。所以如果我們做得比參考答案還好，分數就會 < 1 。我們的目標不是「追平」參考答案，而是「超越」它。這句話會推翻一整類做法。

⚠ 關鍵 2：軟規定違規是「指數」懲罰，碰不得

違規越多，懲罰不是「加上去」而是「指數倍乘上去」（最高 7.4 倍）。也就是說，**哪怕接線跟面積做得再好，只要違規一多，分數立刻爛掉**。把軟規定做到零違規，是底線，不是加分。

⚠ 關鍵 3：大題目決定一切（最重要！）

總分是把每道題的分數加起來，但**每道題的權重是 e^n** （ n = 那道題的積木數，範圍 21 到 120）。這代表大題目的權重**遠遠**輾壓小題目：

21 個積木	權重 ≈ 1 (基準)
60 個積木	權重 $\approx 10^{17}$ 倍
90 個積木	權重 $\approx 10^{30}$ 倍
120 個積木	權重 $\approx 10^{42}$ 倍

📖 白話翻譯

一道 120 積木的大題，重要性是一道 21 積木小題的 10^{42} 倍（這數字比全宇宙原子還多）。意思是：**你在大題目上爛掉，前面小題全部做滿分也救不回來**。所有策略都必須圍著「把大題目做好」打轉。（註：此 e^n 加權的確切公式我們仍要再次跟最新賽規核對，但「大題目壓倒性重要」的結論不會變。）

第 4 章 我們現在有三套做法

我們的專案目前同時養著三條路線，各有強弱。先用比喻認識它們。

做法 A：模擬退火 SA (目前的主力)

 比喻：鐵匠打鐵

鐵匠打鐵時，一開始鐵很燙、可以隨便敲打變形 (高溫 = 大膽亂試，連變差的也接受)，隨著溫度降低，只能做小修正 (低溫 = 只接受變更好的)，最後定型。「模擬退火 (Simulated Annealing, SA)」就是模仿這個過程：先大膽亂試跳出爛區域，再慢慢收斂到好答案。

我們還用了一種叫 **B*-tree (B 星樹)** 的記法來描述「積木怎麼堆」——你可以把它想成一張「擺放食譜」，照著食譜就能算出每個積木的位置。SA 就是不斷修改這張食譜。

做法 B：ML 暖啟動 (AI 猜起點)

 比喻：老師傅的直覺

與其讓鐵匠從一塊冷鐵開始，不如先找個看過上萬個成品的老師傅，讓他先擺一個大概不錯的草稿，鐵匠再從這個草稿開始打。我們訓練了一個 AI 模型，看過範例後直接「猜」一個起點給 SA。(但這個做法現在是壞的——第 6 章詳述。)

做法 C：電靜力法 (目前分數最好)

 比喻：帶電的磁鐵積木

把每個積木想成同性相斥的帶電粒子：它們會自動互相推開 (不重疊)，同時有電線的積木之間像有橡皮筋把它們拉近。讓這個物理系統自己「鬆弛」到平衡，積木就自動排好了。這叫「電靜力法 (electrostatic method)」。

 目前成績

做法 C 目前分數最好 (驗證集 Total Score = 2.966，100% 通過硬規定)。所以它是我們的「保底主力」。

三套做法比較

做法	一句話	強項	弱項
----	-----	----	----

A. 模擬退火	打鐵・亂試後收斂	小題目很準	大題目搜不完 (致命)
B. ML 暖啟動	AI 猜起點	快	現在的版本壞了 (第 6 章)
C. 電靜力法	帶電粒子自己排	大題目也能算、目前分數最好	處理「規定」時較弱

第 5 章 為什麼光靠 SA (打鐵) 贏不了？

這要誠實面對。模擬退火 (做法 A) 在小題目上很好，但在大題目上有一道數學上的牆。

💡 比喻：在黑暗中摸一座天文數字大的迷宮

120 積木的題目有 10^{250} 種擺法。我們的時間預算大概只夠「試」100 萬 (10^6) 種。

$10^6 \div 10^{250} =$ 你只摸過了整座迷宮的 0.000... (後面 244 個零) ...1。

這跟「摸黑找出整個太平洋裡的某一滴特定海水」沒兩樣——再怎麼調打鐵的溫度也沒用，因為樣本數相對搜尋空間根本是零。

⚠️ 兩件事撞在一起 = 為什麼 SA 不可能單獨拿第一

① 大題目正是 SA 最弱的地方 (搜不完)。

② 大題目又是分數權重最重的地方 (第 3 章關鍵 3)。

你最弱的地方，剛好是最重要的地方。這就是為什麼「再調 SA 參數」不會讓我們奪冠——這是數學限制，不是努力問題。

那大題目要靠誰？答案是做法 C (電靜力，能擴展到大題目) 和接下來第 7 章要講的升級版 AI。

第 6 章 我們的 AI (做法 B) 現在哪裡壞了？

這章是整份文件最重要的部分。我們現在的 ML 暖啟動有兩個根本性的錯。

錯誤一：取平均 = 取到爛答案 (Mode Collapse)

我們現在的 AI 是用「回歸」訓練的——簡單說就是讓 AI 去猜每個積木座標的「平均答案」。問題是：好的擺法常常有很多種，而且互相對稱。

💡 比喻：該往左繞還是往右繞？

前面有一根柱子擋路。正確答案有兩個：「往左繞」或「往右繞」，兩個都對。
但如果你叫 AI 去學「這兩個答案的平均」，平均的結果是.....直接往中間走，撞上柱子。
兩個正確答案的「平均」，是一個最糟的答案。

⚠️ 這就是 Mode Collapse (模式崩塌)

floorplanning 的好擺法有大量對稱 (左右鏡射、雙胞胎積木互換都一樣好)。叫 AI 學「平均」，它就會把「靠左的好解」和「靠右的好解」平均成「擠在中間、積木互相重疊的爛解」。
這不是訓練不夠，是方法從根本上錯了。這也解釋了為什麼我們的 AI 暖啟動好像沒什麼幫助。
正解：不要叫 AI 取平均，要叫它「挑一個具體的好答案」(這叫「生成式」模型)。

錯誤二：主辦送的標準答案，我們丟掉沒用

💡 比喻：考古題附了詳解，你卻只看了答案的最後一個數字

主辦的 100 萬筆範例裡，每一筆都附了「完整的擺放食譜」(技術上叫 `tree_sol`，就是第 4 章說的 B*-tree 食譜)。

但我們的程式 (檔案 `ml/data.py` 第 302 行) 寫著一行註解：`tree_sol -- unused` (沒用到)。

我們只拿了「最終座標」來學，把最寶貴的「怎麼擺出來的步驟」整個丟掉了。

★ 為什麼這超級可惜

那 100 萬份「擺放食譜」是接近最佳的標準答案。如果我們讓 AI 直接學「怎麼擺」(學食譜)，而不是「擺完長怎樣」(學座標)，就完全避開了「取平均撞柱子」的問題，而且直接沿用我們現成的工具。大多數參賽隊伍不會發現這個資料可以這樣用——這是我們的秘密武器。

第 7 章 怎麼贏？（奪冠策略白話版）

💡 核心比喻：AlphaGo 怎麼打敗世界棋王

AlphaGo 分兩步變強：

- ① **先模仿**：看幾十萬盤人類高手的棋譜，學會「人類會怎麼下」。
 - ② **再超越**：自己跟自己下幾百萬盤，找到「比人類更好的下法」。
- 我們要對 **floorplanning** 做一樣的事。

四步驟流水線

1 先模仿 讓 AI 看那 100 萬份「擺放食譜」（`tree_sol`），學會擺出「接近標準答案」的擺法。這一步用「**生成式**」模型（會挑一個具體好答案，不取平均）→ 避開撞柱子問題。



2 再超越 用「真正的分數」當獎勵，讓 AI 自己練習、找出**比標準答案更好**的擺法。（標準答案只是「接近最佳」，所以有超越空間 → 做到第 3 章說的 $\text{Cost} < 1$ 。）



3 多猜幾個再精修 請 AI 一次給出好幾個不同的好擺法（候選），每個都用做法 C（電靜力）的物理推擠 **再微調到更緊更短**，最後挑最好的那個交出去。



4 火力集中在大題目 AI 猜一個起點只要 0.1 秒，幾乎免費。所以把所有運算時間和候選數量，全砸在 90–120 積木的大題目上（因為它們決定總分）。

★ 為什麼這套有別人沒有的優勢

- ① 用了別人會忽略的「食譜」資料（`tree_sol`）。
- ② 能「超越」標準答案（多數人只會模仿，停在標準答案的水準）。
- ③ 測試題目和訓練範例來自同一個出題機（主辦明說），所以 AI 學到的東西一定用得上。

📖 兩條腿走路（降低風險）

上面的升級版 AI 很強但工程量大、有風險。所以我們同時保留做法 C (電靜力) 當保底，萬一 AI 來不及完成，至少還有 2.966 的成績。兩條路線共用同一套「物理微調」後端，不會白做工。

第 8 章 接下來誰做什麼

這個專案現在分成「你（人）負責的研究」和「Claude（AI 助手）負責的寫程式」兩部分，可以並行。

你的部分：查資料 + 驗證（用 Google AI Search 和 Connected Papers）

- **先確認地基**：跟最新賽規核對「分數加權公式」到底是不是 e^n 、是 v9 還 v10 版規則。（這是整個策略的基礎，務必先確認。）
- **查方法**：找「生成式拓樸模型」「用獎勵微調超越示範」這兩類最新論文，看別人怎麼做、有沒有現成程式碼可參考。
- 把找到的答案，填回專案文件 `WINNING_STRATEGY.md` 裡標好的空格。

具體的搜尋關鍵字和論文清單，都已經幫你列在 `WINNING_STRATEGY.md` 的第 5 節。

Claude 的部分：寫程式（不需要等你的研究就能開始）

- **T1**：先測現在的 AI 暖啟動到底有沒有用（驗證第 6 章的診斷）。
- **T2**：把丟掉的「食譜」資料 `tree_sol` 撿回來、看懂它的格式。
- **T3**：寫工具確認「食譜」能正確還原成擺法（資料正確性檢查）。
- **T4**：把做法 C 的「物理微調」抽成共用模組（兩條腿共用）。
- **T5**：幫 SA 加一個「低溫精修」模式（配合 AI 給的好起點）。

建議順序：T1 → T2 → T3（先驗證、先解鎖資料），再 T4、T5。

★ 一句話總結整個專案現況

能不能拿第一，關鍵在於「誰真正把那 100 萬份標準答案食譜當成『可以超越的教材』來用」。我們的程式現在正把它丟在第 302 行——把它撿起來，就是別人結構上沒有的東西。

附錄 名詞小辭典

Floorplanning (平面規劃)

決定晶片裡各功能區塊放哪、長怎樣。像在固定盒子裡擺積木。

積木 / 區塊 (block / partition)

晶片裡的一個功能單位，在我們眼中就是一個要擺放的長方形。

HPWL (接線長度)

Half-Perimeter Wire Length。衡量「有連線的積木之間離多遠」，越短越好（省電線）。

netlist (連線表)

記錄哪些積木之間要連線、連得多重要的清單。

約束 (constraint)

擺放的規定。分硬規定（碰到就失敗）和軟規定（違反扣分）。

SA (模擬退火 · Simulated Annealing)

像打鐵：先高溫大膽亂試、再低溫收斂。我們的主力做法 A。

B*-tree (B 星樹)

一種描述「積木怎麼堆」的食譜記法。照食譜就能算出每個積木的位置。

電靜力法 (electrostatic method)

把積木當帶電粒子，靠物理推擠 + 橡皮筋拉力自動排好。做法 C，目前分數最好。

ML / 模型 (machine learning model)

看過大量範例後學會「猜答案」的程式。我們的做法 B。

Mode Collapse (模式崩塌)

AI 把多個正確答案取平均，結果得到一個爛答案（像撞柱子）。我們現在 AI 的致命傷。

生成式模型 (generative model)

會「挑一個具體好答案」而不是「取平均」的 AI。修正 mode collapse 的正解。

tree_sol

主辦範例裡附的「標準答案擺放食譜」。我們現在丟掉沒用——撿回來是奪冠關鍵。

獎勵微調 (reward fine-tuning)

用「真實分數」當獎勵，讓 AI 自我練習、超越標準答案。AlphaGo 變強的第二步。

Cost (成本 / 分數)

每道題的得分，越低越好。可以低於 1（代表比參考答案更好）。

完整技術細節與實作規格，見專案內 [collaborate/WINNING_STRATEGY.md](#)。

本文件為其 0 基礎導讀版。